AI ASSISTANT CODING-23.4

SHIVANI PAMBIDI

BATCH-41

2303A52172

End-to-End Application Development

Task 1 – Problem Identifiication & AI-Assisted Ideation

PROMPT:

Act as a product manager fior a smart city initiative. The city council wants to digitize a manual process. Suggest 3 problem statements. Select one (a Civic Complaint Management System), refiine it, and provide a list ofi key fieatures, target users, and expected impact.

CODE:

Lab23_tests.py U ×

Lab23_tests.py > ...

```python
"""Unit tests for Lab 23 capstone complaint management system."""

import os
import tempfile
import unittest

from Lab23_capstone.service import ComplaintService
from Lab23_capstone.storage import JSONStorage


class TestComplaintService(unittest.TestCase):
    """Validate complaint service behavior and edge cases."""

    def setUp(self) -> None:
        self.temp_dir = tempfile.TemporaryDirectory()
        self.data_file = os.path.join(self.temp_dir.name, "lab23_test_data.json")
        self.service = ComplaintService(JSONStorage(self.data_file))

    def tearDown(self) -> None:
        self.temp_dir.cleanup()

    def test_create_complaint_success(self) -> None:
        complaint = self.service.create_complaint(
            citizen_name="Ananya Rao",
            area="Ward 5",
            issue_type="Garbage Overflow",
            description="Bin has not been cleared for 4 days.",
            priority="High",
```

```python
class TestComplaintService(unittest.TestCase):
    def test_persistence_across_service_reloads(self) -> None:

        self.assertEqual(len(complaints), 1)
        self.assertEqual(complaints[0]["citizen_name"], "Rekha")

    def test_dashboard_metrics_counts(self) -> None:
        self.service.create_complaint(
            citizen_name="Ravi",
            area="Ward 1",
            issue_type="Garbage",
            description="Garbage pickup delayed.",
            priority="Critical",
        )
        self.service.create_complaint(
            citizen_name="Pooja",
            area="Ward 1",
            issue_type="Drainage",
            description="Drain blockage on main road.",
            priority="High",
        )

        self.service.update_status(1, "In Progress")
        self.service.update_status(2, "Resolved")

        metrics = self.service.dashboard_metrics()
```

OUTPUT:

```
Smart City Complaint and Waste Tracking Platform
1. Register new complaint
2. View all complaints
3. Filter complaints
4. Update complaint status
5. Assign staff
6. Dashboard metrics
7. Exit
Select an option: 1
Citizen Name: virat
Area/Ward: warangal
Issue Type (e.g., Garbage, Pothole): garbage
Description: not good
Select priority:
1. Low
2. Medium
3. High
4. Critical
Select option: 3
Complaint created successfully.
[#1] garbage | Area: warangal | Priority: High | Status: Open
Citizen: virat | Assigned: Unassigned | Created: 2026-04-12 13:37:10 | Updated: 2026-04-12 13:37:10
Description: not good
--------------------------------------------------------------------------------
```

JUSTIFICATION:
I chose the Civic Issue Reporting system because it addresses a very common communication gap between residents and municipal authorities. It's a highly relatable problem, and digitizing it ofifiers immediate, measurable benefiits (like tracking visibility), which makes it a perfiect candidate fior an end-to-end sofitware development project.

Task 2 – System Planning & Architecture Design
PROMPT:
Act as a system architect. Design the architecture fior the "CivicFix" complaint system. Suggest firontend, backend, and database components. Defiine the data filow. Propose a basic database schema fior users and complaints.

Architecture & Schema (Text-based):

- **Frontend:** CLI (Command Line Interfiace) fior the prototype (expandable to React.js web app later).
- **Backend:** Python application using modular fiunctions fior core logic (Input handling, Ticket management).
- **Database:** In-memory Python lists/dictionaries (simulating a NoSQL JSON database or SQLite fior production).
- **Data Flow:** User inputs data via CLI -> Backend validates input -> Backend assigns a unique Ticket ID -> Data stored in the dictionary -> User receives confiirmation.
- **Database Schema:**
  - Users: {user_id: INT, name: STR, role: STR [citizen, admin]}
  - Complaints: {ticket_id: INT, citizen_name: STR, category: STR, description: STR, status: STR [Open, In-Progress, Resolved]}

JUSTIFICATION:
I opted fior a modular text-based CLI and in-memory dictionary storage fior this prototype to ensure rapid development and testing within the lab timefirame. The architecture is deliberately decoupled, meaning ifi I want to connect a real SQLite database or build a Web UI later, the core Python logic modules won't need to change.

Task 3 – Modular Code Development with AI
PROMPT :

Write a modular Python program fior the CivicFix system. Include an input handling module to register complaints, core logic to view and assign a status, and data storage using a list ofi dictionaries. Ensure basic error handling and clean structure using fiunctions.

CODE:

```
# CivicFix: Complaint Management System
complaints_db = []
ticket_counter = 1

defi register_complaint(name, category,
  description): """Core logic to register a new
  complaint."""
  global ticket_counter
  ifi not name or not category or not
    description: return "Error: All fiields are
    required."

  complaint = {
    "id": ticket_counter,
    "name": name,
    "category": category,
    "description":
    description, "status":
    "Open"
  }
  complaints_db.append(complaint
  ) ticket_counter += 1
  return fi"Success! Complaint registered with Ticket ID: {complaint['id']}"

defi view_complaints():
  """Core logic to view all
  complaints."""  ifi not
  complaints_db:
```

```
    output += fi"ID: {c['id']} | By:
    {c['name']} | Cat: {c['category']} |
    Status: {c['status']}\n" return output

defi update_status(ticket_id, new_status):
    """Core logic to update complaint
    status."""
    valid_statuses = ["Open", "In-Progress",
    "Resolved"] ifi new_status not in
    valid_statuses:
        return "Error: Invalid status."

    fior c in
        complaints_db: ifi
        c['id'] ==
        ticket_id:
            c['status'] = new_status
            return fi"Ticket {ticket_id} updated to
    {new_status}." return "Error: Ticket ID not
    fiound."

#       Input       handling
simulation   ifi   __name
== "_main_":
    print(register_complaint("John Doe", "Road", "Large pothole on
    Main   St."))   print(register_complaint("Jane   Smith",   "Waste",
    "Garbage not collected.")) print(view_complaints())
    print(update_status(1, "In-Progress"))
```

OUTPUT:

Success! Complaint registered with Ticket ID:
1 Success! Complaint registered with Ticket
ID: 2
--- All Complaints ---
ID: 1 | By: John Doe | Cat: Road | Status:
Open ID: 2 | By: Jane Smith | Cat: Waste |
Status: Open

Ticket 1 updated to In-Progress.

JUSTIFICATION:

I structured the code using clearly defiined, single-responsibility fiunctions (register_complaint, view_complaints, update_status). This modular approach ensures that input validation and business logic are separated firom the data layer (complaints_db). It makes the code much easier to read, test, and debug.

Task 4 – AI-Assisted Testing, Debugging & Optimization

PROMPT:

Review the Python code fior the CivicFix system. Generate 3 test cases. Identifiy potential bugs (e.g., handling invalid ticket IDs during status updates). Provide an optimized version ofi the code that uses try-except blocks where necessary and improves input validation.

Test Cases & Bug Identifiication:

1. **Test Case 1 (Valid Input):** Register complaint with valid strings. *Result: Pass.*
2. **Test Case 2 (Invalid Ticket ID):** Try to update status ofi Ticket ID 99 or passing a string instead ofi an int. *Result: Bug identifiied - code fiails silently or crashes ifi a string is passed instead ofi an integer ID.*
3. **Test Case 3 (Empty Input):** Pass empty strings to register_complaint. *Result: Pass (handled by existing basic validation).*

OPTIMIZED CODE:

```
complaints_db = []
ticket_counter = 1

defi register_complaint(name: str, category: str, description: str) ->
  str: global ticket_counter
  try:
    # Optimization: Stripping whitespace
    name, category, description = name.strip(), category.strip(),
    description.strip() ifi not name or not category or not description:
      raise ValueError("Fields cannot be empty or just spaces.")

    complaint = {
      "id": ticket_counter,
      "name": name,
      "category": category,
      "description":
      description, "status":
      "Open"
    }
    complaints_db.append(complaint)
```

```
    ticket_counter += 1
    return fi"Success: Ticket ID {complaint['id']}
  created." except Exception as e:
    return fi"Error registering complaint: {e}"

defi update_status(ticket_id: int, new_status: str) ->
  str: valid_statuses = ["Open", "In-Progress",
  "Resolved"] try:
    ticket_id = int(ticket_id) # Type enfiorcement
    optimization ifi new_status not in valid_statuses:
      raise ValueError("Invalid status provided.")

    # Optimization: using next() with a generator expression fior fiaster lookup
    O(N) complaint = next((c fior c in complaints_db ifi c["id"] == ticket_id),
    None)
    ifi complaint:
      complaint["status"] = new_status
      return fi"Success: Ticket {ticket_id}
    updated." else:
      return "Error: Ticket ID not
  fiound." except ValueError as ve:
    return fi"Input Error:
  {ve}" except Exception
  as e:
    return fi"System Error: {e}"
```

OUTPUT:

(Testing Optimized Code)
Output fior update_status("abc", "Resolved"): Input Error: invalid literal fior int() with base 10: 'abc'

JUSTIFICATION:
The initial code was firagile regarding data types. Ifi a user entered "one" instead ofi 1 fior an ID, the system would crash. I utilized AI to help implement stricter type hinting, .strip() fior input sanitation, and comprehensive try-except blocks.

Task 5 – Documentation, Version Control & Refilection

PROMPT:

Generate a concise project README.md (Setup + Usage) fior the CivicFix app. Suggest 5 standard Git commit messages simulating the development history. Finally, write a short refilection on how AI helped, limitations fiaced, and fiuture improvements.

README Snippet:

CivicFix: Smart City Complaint System
A lightweight CLI application fior reporting and tracking civic issues.

Setup
1. Ensure Python 3.8+ is installed.
2. Clone the repository: `git clone https://github.com/...`
3. Run the application: `python app.py`

Usage
- Developers can import `register_complaint` and `update_status` firom `app.py`.
- Data is stored in-memory per session.

Simulated Git Commit Messages:

1. init: setup initial project structure and README
2. fieat: add register_complaint fiunction and core data dictionary
3. fieat: implement ticket status update logic
4. fiix: add input sanitation and try-except error handling
5. docs: add python docstrings to core logic fiunctions

Refilection Report:

- **How AI Helped:** AI was incredibly usefiul fior overcoming "blank page syndrome." It rapidly generated the boilerplate logic and provided excellent suggestions fior edge-case testing (like handling string inputs fior integer IDs). It also saved time writing the standard README markdown.
- **Limitations Faced:** While the AI wrote the code quickly, I had to be very specifiic in my prompts to ensure the code remained modular. Initially, the AI tried to put everything into one massive while True: loop, which violated the

intervene to fiorce fiunction-based modularity.

- **Improvements Possible:** In the fiuture, this project should be upgraded firom in-memory lists to an actual database (like SQLite or PostgreSQL). Additionally, attaching a web- based firontend using a firamework like Flask or FastAPI would make it a truly complete, user-fiacing end-to-end application.

JUSTIFICATION:

Proper documentation and simulated version control are what separate a lab script firom profiessional sofitware. Using AI to generate the README and commit history taught me standard fiormatting conventions. The refilection highlights a crucial lesson: AI is a powerfiul assistant, but the developer must remain the "architect" to ensure the output aligns with sofitware engineering best practices (like modularity).